

Staff-fix-plan

/plan Master Architecture, Workflow & Multi-Tenant Refactor for SPIN Platform

Execute a comprehensive end-to-end refactor of the SPIN platform. All changes must align strictly with the existing Firestore database schema, FastAPI endpoints, and React component structures.

CRITICAL SCHEMA CONSTRAINT: CANONICAL DEPARTMENT ENUMS

DO NOT introduce arbitrary display strings for departments (e.g., "Water Supply Operations", "Electrical Board") into database fields or API payloads.

All department filtering, routing, and role assignments must strictly bind to the exact enum keys already defined in the schema and `models.py`:

- `category / department_id` enum keys: `water`, `electricity`, `roads`, `garbage`, `drainage`, `other` (extend only with standard `snake_case` keys if additional municipal departments are declared in `models.py`).
 - Any human-readable display label (e.g., "Water Supply", "Public Works") must exist exclusively in a frontend UI translation mapping dictionary, never in Firestore or JWT claims.
-

1. Authentication, Storage & Database Integrity

A. Resolve Dual-Token Desync

- **Files:** `frontend/dashboard/src/services/demandService.ts` & `authService.ts`
- **Issue:** `fetchDemands()` uses `auth.currentUser?.getIdToken()`, while mutations (`castVote()`, `submitRequestToBackend()`) rely on `localStorage.getItem("citizen_token")`, causing stale 401 Unauthorized errors on token refresh.
- **Fix:** Standardize all authenticated requests in `demandService.ts` to retrieve fresh tokens dynamically via `await auth.currentUser?.getIdToken()` with fallback to cached tokens only when offline.

B. Replace Mocked Image Uploads with Firebase Storage

- **File:** `frontend/dashboard/src/services/demandService.ts` (`uploadEvidenceToBackend`)
- **Issue:** Currently returns transient client blobs (`URL.createObjectURL()`).
- **Fix:**
 - Import `getStorage`, `ref`, `uploadBytes`, and `getDownloadURL` from `firebase/storage`.
 - Upload evidence files to `demands/{user_id}/{timestamp}_{filename}`.
 - Store the resulting HTTPS download URLs in the demand document's `media_urls` array.

C. Update Pydantic Schemas & Sanitization

- **File:** `backend/spin_agents/models.py`
 - Set `vote_count: int = 0` (default was 1).
 - Add missing fields to `DemandSchema`: `address`, `pincode`, `landmark`, `request_type` (default `"maintenance"`), `reason`, `intended_beneficiaries`, `media_urls` (list of strings), and `timeline` (list of dicts).
- **File:** `backend/spin_agents/runner.py`
 - Replace fallback `"Unknown"` strings for `district` and `state` with `None` so null values are stored cleanly.
- **File:** `backend/spin_agents/services/demand_service.py`
 - In `persist_demand_to_db()`, replace nested `firestore.SERVER_TIMESTAMP` sentinels inside the `timeline` array with ISO 8601 UTC strings (`datetime.now(timezone.utc).isoformat()`).
 - Denormalize author display name: Fetch the submitter's `name` from `users/{uid}` and store it as `author_name` on the demand document.

D. Server-Side User Filtering (Fix O(N) Query)

- **Backend:** In `backend/spin_agents/routers/demand_router.py`, add support for `GET /api/demands?author_user_id={uid}` using Firestore's `.where("author_user_id", "==", uid).order_by("created_at", direction=Query.DESENDING)`.
 - **Frontend:** Refactor `demandService.getMyRequestsFromBackend()` to pass `author_user_id` as a query param rather than filtering client-side.
-

2. Frontend Routing & Just-In-Time (JIT) Voting Workflow

A. Decouple Route Guards & Enable Real URL Routing

- **File:** `frontend/dashboard/src/App.tsx`
- **Changes:**
 - Remove `"citizen"` (Feed) and `"citizen-detail"` from the unauthenticated route redirect list.
 - Restrict redirect guards strictly to protected actions: `"citizen-raise"` and `"citizen-profile"`.
 - Implement real parameterized routes using `react-router-dom`:
 - Public Feed: `/feed`
 - Public Demand Detail: `/demand/:demand_id`
 - Track Requests: `/track`
 - Raise Demand: `/propose`

B. Just-In-Time (JIT) Voting

- **Files:** `CitizenPortalHome.tsx`, `DemandDetail.tsx`, and `App.tsx`
- **Workflow:**
 - Any anonymous visitor can view `/feed` and `/demand/:demand_id` with full context, map markers, and vote counts.

- When an unauthenticated visitor clicks "Vote":
 1. Persist the current demand ID to `localStorage.setItem("pending_vote_demand_id", demand_id)`.
 2. Open the authentication modal or redirect to login.
 3. Post-login, check `localStorage.getItem("pending_vote_demand_id")`, auto-dispatch `POST /api/demands/{id}/vote`, and remove the key.
-

3. UI/UX Overhaul: Feed, Detail & Tracking Views

A. Citizen Feed (`CitizenPortalHome.tsx`)

- **Remove Staff Icon:** Remove the staff login shortcut icon from the citizen-facing top navigation bar.
- **Card Data Binding:**
 - Replace raw category text in card headers with the actual user `title` or truncated `english_translation`.
 - Render the category as a subtle tag/pill using standard enums (`water`, `electricity`, etc.).
 - Map raw status enum `gathering_support` to a styled badge: "Gathering Support".
 - Add a styled progress bar showing `vote_count` / `vote_threshold` (e.g., 12 / 100 votes).
- **Map Synchronization:**
 - Plot markers for all active demands in the current feed query.
 - Implement two-way hover/click highlighting between cards and map markers.

B. Citizen Track Page (`CitizenRaiseRequest.tsx` / `MyRequests.tsx`)

- **Remove Internal Leaks:** Completely delete the bottom metadata bar displaying `Routing Authority`, `Database Persistence: Authoritative SQLite`, and `Cloud Warehouse: Pending Scheduled Batch`.
- **Data Cleanup:** Format locations cleanly (`Pune`, `Maharashtra`); omit missing landmarks or districts instead of rendering "None" or "Unknown".
- **Mobilization:** Add a "Share to WhatsApp" button next to "View Details" linking to `https://<domain>/demand/:demand_id`.

C. Demand Detail Page (`DemandDetail.tsx`)

- **URL Parameter Handling:** Ensure `DemandDetail` reads `useParams<{ demand_id: string }>()` and triggers `demandService.getRequestDetailFromBackend(id)` on mount if data is not already in memory.
- **Public Mobilization Panel:** Add a primary "Back This Demand / Vote" button with live status (`Supported` ✓ if already voted), a vote progress bar, and a "Share to WhatsApp" action.
- **Mini-Map:** Replace raw coordinate floats (`18.62...`, `73.91...`) with an embedded mini-map centered on `[latitude, longitude]`.
- **Multi-Stage Civic Stepper:** Replace the raw UTC timestamp bullet with a 5-stage vertical or horizontal progress stepper:
 1. `gathering_support` (Active)

2. `under_review`
3. `field_survey`
4. `approved_for_budget`
5. `fulfilled`

- **Evidence Gallery:** Render thumbnail cards for URLs in `media_urls` with lightbox enlargement; show an empty state when none exist.

4. Hierarchical Multi-Tenant RBAC & Staff Management

Implement a 3-tier geographic and domain-scoped administrative hierarchy:

[Tier 0: Platform System Admin] — Global configurations & infrastructure

|



[Tier 1: State Admin] — Regional oversight & District Admin provisioning

|



[Tier 2: District System Admin] — District site manager, technical support & staff provisioning

|

| — Provisions staff across exact category enums ('water', 'electricity', etc.)



| — Municipal Execution Silos

| For each category: water, electricity, roads, garbage, drainage, other |

| — Department Policymaker (1 per category) |

| — Department Officer (1–2 per category) |

| — Field Officers (Multiple per category, ward-mapped) |

A. Scoped Firestore Schema

Update staff user records in the `users` collection:

```
{  
  "uid": "staff_uid_here",
```

```

"name": "Staff Member Name",
"email": "staff@civic.gov.in",
"role": "department_officer", // Options: "platform_admin", "state_admin",
"district_admin", "policymaker", "department_officer", "field_officer"
"state_id": "maharashtra",
"district_id": "pune",
"department_id": "water", // MUST match backend category enum: 'water',
'electricity', 'roads', etc.
"assigned_wards": ["Ward-14", "Ward-15"],
"status": "active"
}

```

B. Administrative Capabilities

1. **Tier 0 (Platform Admin):** System health, API keys, AI model orchestration.
2. **Tier 1 (State Admin):** Provision Tier 2 District Admins; set regional vote thresholds.
3. **Tier 2 (District System Admin):**
 - **Zero-Trust Staff Onboarding:** Provisions accounts via email/SMS invitation links (never handles plain-text passwords). Assigns staff to specific `district_id`, `department_id`, and `assigned_wards`.
 - **Site & Tech Support:** Handles staff lockouts, hardware GPS validation errors, and offline PWA sync failures.
 - **Category Re-Routing Console:** Allows manual re-assignment of demands flagged as mistargeted by the AI pipeline across the valid department category enums.

5. Department & Field Officer Workspaces

A. Department Officer Workspace (`StaffDashboard.tsx`)

- **Live Firestore Queries:** Replace stubbed empty arrays in `backend/spin_agents/routers/staff_router.py` :
 - `GET /api/staff/demands/queue`: Query demands where `district_id == current_user.district_id AND category == current_user.department_id AND status IN ["gathering_support", "under_review"]`.
- **Queue Division:**
 - *Threshold Trigger Queue*: Demands where `vote_count >= vote_threshold`.
 - *Emerging Queue*: Demands with high vote velocity for proactive monitoring.
- **Survey Dispatch Modal:** Provide an action to assign a demand to an active field officer matching the same `department_id` and ward. Update demand status to `"field_survey"` and append to `timeline`.
- **Split-Screen Feasibility Review:**
 - *Left*: Citizen submission details, native text, Bhashini translation, citizen photos.
 - *Right*: Field survey results (feasibility checklist, notes, EXIF hardware-verified photos).
 - *Actions*: One-click buttons to "Approve to Policy", "Request Re-Inspection", or "Reject with Reason".

B. Field Officer Workspace (`FieldDashboard.tsx`)

- **Responsive Split Layout:** Convert the narrow mobile container on desktop screens to a full-width split-pane layout: Queue List (30% width) and Interactive Map (70% width) rendering side-by-side. On mobile screens, retain the List/Map toggle.
- **Real Map Pin Binding:** Remove generic POIs; center the map bounding box dynamically around the officer's assigned demands using their exact `latitude` and `longitude`.
- **Survey Submission Drawer:**
 - Query assignments where `status == "field_survey"` AND `assigned_officer_id == current_user.uid`.
 - Form items: Camera capture widget (validates on-site EXIF GPS metadata against demand coordinates), feasibility checklist (access, clearance, safety), and survey notes.
 - Submission updates demand status to `"feasibility_reported"`, appends to `timeline`, and frees the ticket for Department Officer review.